

er using other shadowing issues as an example (example)

Product Leadership

The separate page Product Leadership covers how to be an effective leader in the product management organization.

SECTION: product/product-processes/_index.md

{{% include "includes/product-handbook-links.md" %}}

Our Product philosophy

As a Product Organization, we work to create a flexible yet concise product development framework for developing products that customers love and value. The Product Principles section is where you can learn about our strategy and philosophy regarding product development, here we discuss the processes we use tactically.

Product Development Flow

Introducing changes requires a number of steps, with some overlap, that should be completed in order. GitLab follows a dual-track product development flow spanning product, engineering, UX, and quality. We use GitLab to power product development flow. When changes are released, we follow the release post process to communicate externally about new capabilities.

This process should be both up front and on an on-going basis when building features.

The Importance of Direction

Documenting a Section, Stage, Group and Category direction is critical to communicating where we are heading and why to all of our stakeholders. We document our direction in direction pages. Read more about related processes under Planning and Direction.

Understanding Milestones and Releases

- Interpreting release dates clarifies how product teams use milestones and labels to indicate the likelihood of feature delivery within certain time frames.
- The release definitions are maintained by the Engineering Team and we run the end of each Milestone on the release date.

Relevant links

- Engineering release definitions
- Feature flag lifecycle
- Product Launch process

Communication

Product Org Communication Touchpoints

In November 2024 we adjusted our communicated touch points based on feedback from our FY25-Q2 Engagement Survey, AMAs, skip levels, and 1:1s. The two key areas we are prioritizing:

- Embedding more asynchronous touchpoints to reach team members across time zones
- Implementing additional opportunities to cascade information from E-Group and PLT

Synchronous Product Org Communication Touch Points

- Product All-Team Meeting: We have a quarterly synchronous meeting for the Product Org. As of Nov 2024, we are expanding on the existing content and structure to ensure key strategic updates from each PLT department head are effectively cascading through the organization regularly.
- Product Key Review: We follow the company wide key-review format which covers key updates to the product division and is shared with cross-functional stakeholders.

Asynchronous Product Org Communication Touch Points

- CPO Fortnightly Update: David posts a slack message in product-private every two weeks. This is a way for him to share what is top of mind for E-group and across the product team.
- New Hire Intros: New hires to the product org are encouraged to post their introduction in the new-team-members slack channel. Once per month, a member of our Product Leadership Team will post a consolidated list of new hires and their intros into the product-private slack channel.
- Monthly Showcase: On a 4-week cadence we have an async monthly showcase, facilitated in the product-private channel, where people can share what they're working on with the rest of the Product Org. This is a low-key competition with prizes to incentivize folks to contribute videos. Team members can add videos throughout the month, then the entire Product Org can vote on their favorite video via google form. At the end of the 4-week period, there will be a prize for the winner, announced in David's fortnightly update. In the case of a tie in number of views, David will make the final call on who wins. We hope that this encourages team members to share their wins and exciting developments in their feature areas! Note: We're doing this on a 4-week cadence (rather than calendar month) so that it aligns with David's fortnightly updates, which are posted every other week. This should still be roughly aligned to the calendar month.

Other Communication Channels

- Product Function Issue Tracker - please use confidential issues for topics that should only be visible to team members at GitLab.
- Product Function GitLab Group - @gl-product
- Product Management GitLab Group - @gl-product-pm
- [Chat channel - product-private]. This is a private slack channel for members of the product-org; new team members are automatically added to this channel as part of onboarding.
- Chat channel - product. All GitLab internal team members can use the product channel for any product-related questions but you'll also find more direct assistance in the various Product Group channels.

Communicating with the Entire Product Management Function At Once

When communicating change or a request for action to the entire product function, utilize the following levels and corresponding activities.

Level	Description	Activities
One	Suggestion for review from interested PMs and FYI	Post MR/issue in product
Two	Request for action from all PMs	Post in product and mention @gl-product-pm in MR/issue with specific action instructions.
Three	Confirmation of understanding	Post in product and mention @gl-product-pm; checkbox for each @gl-product-pm member in an MR/issue description to confirm; assign MR/issue to all @gl-product-pm members

Internal and external evangelization

Before shipping a new or updated feature, you are responsible for championing it, both internally and externally. When something is released, the following teams need to be aware of it as they will all need to do something about it:

- Marketing: depending on the importance of the feature, we need the help of marketing to promote this feature on our different communication channels.
- Sales: sales needs to know what's new or changed in the product so they can have better arguments to convince new or existing customers during their sales process.
- Support: as they are in constant contact with our users and customers, support should know exactly how our products work.

You can promote your work in several ways:

- start with documenting what will be released and share this documentation with the different teams
- schedule meetings, if you think it's important, with the teams listed above.

When referencing issues in written communication using just the issue number 123456 and a link is not low-context communication. Instead use the title of the issue and the link or the issue number and description of the problem that issue will solve:

- Good: We will next be working on Detect and display code coverage reports on MR. OR We will next be working on gitlab21549 which will help developers view code coverage reports directly in GitLab instead of losing context by looking in another tool while reviewing an MR.
- Avoid: We will next be working on 21549..

In order to support findability and to clearly articulate when we change our minds especially when it comes to product direction, category changes, shifts in investment themes, or priorities for engineering, Product Managers must evangelize these changes in multi-modal communication channels to ensure our users and customers aware.

Some internal methods for communication include:

- Sharing the updates various product-based Slack channels such as: product, s, g, or f Slack

channels

- Cross-posting changes in direction or categories into customer-success and if they impact use cases tag @cs-leadership for awareness
- Recording a quick video and sharing with Customer Success that discusses direction updates. Use sync meetings as needed to facilitate efficient communication.
- Collaborate with the Field Communications team to determine if a larger internal communications plan/approach is necessary for the Field (Sales, Customer Success, Channel & Alliances) team.
- Aggregating and sharing highlights of monthly direction page updates at the Section-level across the organization

External channels for consideration linking direction pages to:

- Twitter, LinkedIn, or other social accounts
- Sharing outreach emails via account teams
- Recording walkthroughs on Unfiltered and promoting on social accounts
- Writing a blog about the changes, if they are significant or disruptive

Writing to inspire action

As a PM, it is important to remember a bias towards action (and other value actions like sense of urgency, make a proposal, boring solutions, write things down, don't wait, and make two way doors decisions

which enables PMs to drive an async discussion to being action oriented. Every time you write a comment or create an issue ask yourself: Will this allow us to take an action and move us forward?

Writing about features

As PMs we need to constantly write about the features and upgrades we ship: in a blog post, internally to promote something, and in emails sent to customers. There are some guidelines that one should take into account when writing about features, the most important being a clear communication of the problem we're solving for users.

When writing about a feature, make sure to cover these messaging guidelines which help produce clear internal and external messaging. Please also keep in mind that we should avoid using acronyms that others may not recognize, such as "MVC" for Minimal Valuable Change. For more guidance you can visit our writing style guidelines.

Let's highlight the messaging guidelines mentioned above with a concrete example, Preventing Secrets in your repositories, that we shipped in 8.12.

- Start with the context. Explain what the current situation is without the feature. Describe the pain points and connect back to our Value Drivers (in this case Reduce Security and Compliance Risk).

- > It's a bad idea to commit secrets (such as keys and certificates) to your
- > repositories: they'll be cloned to the machines of anyone that has access to the

- > repository. If just a single one is insecure, the information will be
- > compromised. Unfortunately, it can happen quite easily. You write
- > `git commit -am 'quickfix' && git push` and suddenly you've committed files that
- > were meant to stay local!

- Explain what we've shipped to fix this problem.

- > GitLab now has a new push rule that will prevent commits with secrets from entering the repository.

- Describe how to use the feature in simple terms.

- > Just check the checkbox in the repository settings, under push rules and

- > GitLab will prevent common unsafe files such as `.pem` and `.key` from being committed.

- Point to the documentation and any other relevant links (previous posts, etc).

Here are some additional examples of well written release blog posts for inspiration:

- Issue Board Work In Progress Limits
- Parent-Child Pipelines
- Drag-and-drop Design badges
- Render charts in GitLab issues using a Grafana URL

Recording videos to showcase features

In addition to the written medium, video is an important medium that caters to the different goals you are trying to accomplish and learning styles of your audience.

Depending on the type of video you are recording, there are some guidelines to keep in mind.

As our documentation guidelines actively encourage linking video content, please consider following the Documentation Style Guide section on language, and working with your technical writing team to include links to your speed runs, walk-throughs and demos at relevant locations in the product documentation.

Using GIFs

Animated GIFs are an awesome way of showing of features that need a little more than just an image, either for marketing purposes or explaining a feature in more detail. Checkout our guide to Making Gifs!

Speed Run

Speed runs are informal videos meant to focus on a single workflow and the experience for performing that workflow. It should not require much planning and is typically short in duration (less than 5 min.). This video type is meant to inform and not necessarily to influence buyers.

Examples:

- GitLab Unfiltered Speed Runs Playlist
- Remove docker images via CI/CD speed run

Demo

Demos are scripted recordings meant to influence buyers. Generally has higher production value and typically involves both a slide-style presentation and/or live screen-sharing. Duration varies depending on the topics being covered.

Examples:

- GitLab for the Enterprise Demo

Walk-through

Product walk-throughs are informal videos meant primarily for an internal audience as a recorded, visual form of product critique. Walk-throughs typically focus on the user experience across categories and workflows within a Product Manager's product scope. There are particular benefits to walk-throughs which span product hierarchy boundaries (multi-category, multi-stage, multi-section) as they help highlight disjointed experiences across our single-application.

Walk-throughs are typically longer in length as they cover more ground and often involve some "live" troubleshooting and are best performed with no planning. Use the Product walk-through issue template when creating a walk-through.

Examples:

- Auto DevOps setup and usage walk-through

QA Release Candidates on staging and elsewhere

After the feature freeze, it's expected of each product manager to test their own features and perform quality assurance to the best of their ability and follow up where necessary.

Product managers can use the staging environment once the release managers have deployed a release candidate (RC) to staging.

Release managers should post in the product channel in Slack that a new release candidate is available. Product managers can also use other environments as needed, such as GitLab provisioned on Kubernetes with GKE.

Feature assurance

Before a new feature is shipped, the PM should test it out to make sure it solves the original problem effectively. This is not about quality assurance (QA), as developers are responsible for the quality of their code. This is about feature assurance (FA). FA is necessary because sometimes there are misunderstandings between the original issue proposal and the final implementation. Sometimes features don't actually solve the intended problem, even though it seemed like it would, and sometimes solutions just don't feel as

useful as intended when actually implemented.

If you can test out the feature during development, pulling down branches locally (or with a review app!), that's great. But sometimes it's not feasible to test a feature until it's bundled into a release candidate and deployed to GitLab.com. If so, make sure to test out features as soon as possible so any new issues can be addressed before final release. Also, take the FA cycle into account when scheduling new milestone work.

If you are looking to test code that has not been merged to GitLab.com or is not yet part of an RC, you can pull the branch down locally and test it using the GitLab Development Kit (GDK).

Dealing with security issues

Quality Engineering Managers (QEM) are the DRIs for prioritizing bugs. These include security issues which are prioritized in conjunction with the security team. Product Managers must work with their QEM to set Milestones for issues marked with the bug::vulnerability type label to guarantee they are shipped by their due date, as defined in the Security Team process.

While Product Managers are the DRIs for milestone planning, they must respect the prioritization order for bugs and maintenance issues as determined by their QEM and EM, respectively. As such they should deeply understand the implications and risks of security-related issues and balance those when prioritizing a milestone work. Addressing a serious security issue by its due date may require temporarily adjusting the desired work type ratio for one or more milestones. Priority labels and Due Date designations for security issues should never be modified by Product Managers as they are directly managed by the Security Team and used to track metrics and progress.

Foundational Requirements

When thinking about new features, we must not only think about the functional requirements of a feature (defining what the feature will do), but also to think about foundational requirements (defining how the feature works). At the highest level, foundational requirements define items such as performance, scalability, compatibility, maintainability and usability characteristics of a feature. It is important to have foundational requirements in place up front, as this is much easier than trying to add them later and change expectations, or break existing workflows. Our definition of done contains specific areas of consideration that are required for the acceptance of new contributions.

For an in depth review of foundational requirements (often referred to as non-functional requirements), see this resource.

To deliver features, we must have both functional and foundational requirements defined.

Introducing application limits

To enhance availability and performance of GitLab, configurable limits should be put in place for features which utilize storage, or scale in a manner which could impact performance. For example, we limit the number of webhooks per project, and we allow admins to set rate limits on

raw endpoints. These limits ensure more consistent performance, reduce the likelihood of outages, and offer admins tools to limit abuse or enforce specific standards. While these limits can be configurable, sensible default limits should be defined for our GitLab SaaS and GitLab dedicated offerings.

There is a guide about developing application limits in the GitLab Docs.

When implementing application limits

See Rate Limiting::Managing Limits.

Managing data lifecycle and growth

As we continue to scale our product, we need to consider the amount of data being stored for new features. Data storage is not an infinite resource, so we should think carefully about what data needs persistent storage to provide the desired user experience. We also need to consider the cost implications around data storage. Everything we store impacts our bottom line, and we should therefore be careful to ensure we are only storing necessary data for well thought out time-frames. We are working on defining a sustainable data retention policy, and will iterate on this section as more general guidelines are developed.

Data storage comes in three main forms for GitLab -- object storage, database storage, and Git repository storage. While we have dedicated teams devoted to ensuring we can scale these storages appropriately, it is in our best interest to only store what is required for a feature to perform as intended. Additionally, there are situations where storage should be subject to data retention policies.

Considerations around data storage

When evaluating feature data storage, the following data storage topics should be considered.

- What quantity data needs to be stored? - What amount of data will need to be stored for the feature to function as intended. Is this level of data storage bounded, or is there a potential for unbounded growth? Unbounded growth should be avoided if possible.
- How long should data be retained? - We should consider carefully the need to store data indefinitely. For many features, removing certain data after a specified time period won't impact the functionality of the feature. In these instances, we should put retention policies in place. These retention policies should have a sane default value which is considered best practice for operating the feature long term. Note: it is easier to iterate toward longer data retention time frames, but far harder to reduce retention time frames. Consider starting out with a conservative time frame.
- How often will this data be accessed? - Much like the quantity of data stored can lead to scalability issues, so can the increased load on the data stores when the data is accessed frequently. There are ways to ease the burden on our infrastructure by properly forming queries, caching often used data, or carefully considering how repository data is accessed. If there are questions, consider reaching out to the Database Group or the Git Group for assistance.

A good example where we've successfully evaluated data storage is our CI/CD Artifacts. We've set some sane default values for both maximum artifact size and for default artifacts

expiration, while making these both configurable for administrative users.

Cross-stage features

See this page for details on working across stages at GitLab.

Stages, Groups, and Categories

Stages, groups, and categories serve as a common framework for organizing and communicating the scope of GitLab.

How to work as a PM

If you follow the principles and workflow above, you won't be writing long, detailed specs for a part of the product for next year. So how should you be spending your time?

Invest the majority of your time (say 70%) in deeply understanding the problem. Then spend 10% of your time writing the spec for the first iteration only and handling comments, and use the remaining 20% to work on promoting it.

A problem you understand well should always have a (seemingly) simple or obvious solution. Reduce it to its simplest form (see above) and only ship that.

Prioritization

See the Cross-Functional Prioritization page for more information.

Prioritization Framework

```
{{% include "includes/master-prioritization-list.md" %}}
```

```
{{% include "includes/engineering-allocations.md" %}}
```

```
{{% include "includes/feature-change-locks.md" %}}
```

Please also note the corresponding Engineering handbook section about the relative importance and prioritization of availability, security, and feature velocity. To ensure we're providing an appropriate focus on security, data loss, and availability, PMs should consider:

- tracking the appropriate labels for each prioritization category: Use a standing item to discuss these issues with an engineering manager and ensure you understand the impact of related issues in your area before planning a release.
- optimizing for quality once a merge request is ready for review: This means ensuring that Engineering has sufficient time to meet our definition of done - including a high-quality code review - without cutting corners to get something into production.

Prioritization sessions

To help PMs plan, stage group stable counterparts can participate in prioritization sessions.

They serve mainly as an internal sensing mechanism for PMs to make more informed prioritization decisions for different planning horizons. Usually, teams focus on the product releases horizon, but can also focus on the FY themes or strategy horizons. This group exercise also boosts team morale, improves communication and empathy, and broadens individual's perspectives. Besides, it can be a more informal and joyful way of connecting the team and discussing work.

The output of these sessions is a priority matrix that shows the relative priority of a set of items based on two weighted criteria. Generally, the criteria are importance and feasibility, each one visualized as an axis of the matrix. You can change the criteria depending on the planning horizon or goals. To better understand how the sessions work, see an example mural and session recording.

Always consider asynchronous sessions first, in an effort to be more inclusive and respectful of others time. That said, if possible, synchronous sessions can be ideal, as they allow limiting the time spent and make great use of the activities' momentum for a more efficient discussion and voting.

Use our Mural template for prioritization sessions, built for product releases but adaptable for other planning horizons or criteria.

<details markdown="1">

<summary markdown="span">Process template</summary>

Adapt this process as needed, and consider changing it to an asynchronous mode of communication. For example, participants can review the items async, add questions as comments in Mural, and vote using dot voting or in voting sessions held on different days for each criterion.

1. Before:

1. The facilitator creates a mural from our template for prioritization sessions, with the stage group and milestone in its name.

1. The facilitator invites the stage group counterparts for a 50-minute call, scheduled sometime before the team finalizes the release scope (see the product development timeline). Includes the URL of the mural and planning issue in the event description.

1. The facilitator shares the preparation work with the participants, preferably in the group's planning issue (see the template after this list and an example).

1. Participants do the preparation work (see the template after this list).

1. During (see an example session recording):

1. The facilitator starts recording the call.

1. Present: For each participant, the facilitator sets the timer for 10 minutes (adapt per the no. of participants). A participant then presents their issues, preferably using the RICE framework. Only after the participant presents all issues should other attendees ask questions. Once in a while, the facilitator announces how much time remains. When the timer goes off, repeat this for another participant.

1. Vote: After all participants have presented, the facilitator runs two voting sessions: first for importance, and then for feasibility. Each participant has 5 votes (adapt per the no. of issues). The facilitator sets the timer for 2 minutes, repeating for each voting session.

1. Visualize: Review your voting session results and everyone helps place the stickies on the matrix, depending on their number of votes for each criterion.

1. If there's still time, discuss the most-voted issues as a group.

1. After:

1. The facilitator uploads the recording to GitLab Unfiltered, sets its visibility (see SAFE framework), adds to relevant playlists, and includes the URL of the mural and planning issue in the description.

1. The facilitator shares the recording URL and voting results in the planning issue, preferably with a screenshot of the matrix and links to the highest voted issues (see an example).

<details markdown="1">

<summary markdown="span">Preparation work template</summary>

markdown

:map: Prioritization session

@-mention participants for our prioritization session, here's the Mural for us to add the issues we want to see in MILESTONE. I scheduled our 50-minute session for DATE.

1. Add your issues to the Mural before the call. Let's try to limit to 5 issues per person, so it's easier to vote on them and keep things focused. You can find instructions on how to add them in the "Outline" panel on the right side of the Mural UI.

1. Try not to add Security or Availability issues. This is also noted in the product processes page, as those issues have forced prioritization with SLAs/SLOs.

1. If you can, mark issues that appeared in previous sessions by changing their sticky color to orange.

Thanks and see you soon :bow:

</details>

</details>

Using the RICE Framework

RICE is a useful framework for prioritization that can help you stack rank your issues. The RICE framework is a great tool for prioritizing many issues that seem to be of equal value at first glance. In order to drive clarity and alignment in the prioritization of work across the entire DevOps platform, and to help prioritize items that may compete for resources from different teams, we have set a standard for the RICE factors so all prioritization decisions based on RICE are using the same metric.

Reach How many customers will benefit in the first quarter after launch? Data sources to estimate this might include qualitative customer interviews, customer requests through Support/CS/Sales, upvotes on issues, surveys, etc.

Higher reach means a higher RICE score:

- 10.0 = Impacts the vast majority (~80% or greater) of our users, prospects, or customers
- 6.0 = Impacts a large percentage (~50% to ~80%) of the above
- 3.0 = Significant reach (~25% to ~50%)

- 1.5 = Small reach (~5% to ~25%)
- 0.5 = Minimal reach (Less than ~5%)

Impact How much will this impact customers and GitLab? Impact could take the form of increased revenue, decreased risk, and/or decreased cost (for both customers and GitLab). This makes it possible to compare revenue generating opportunities vs. non-revenue generating opportunities. Potential for future impact should also be taken into account as well as the impact to the GitLab brand (for example unlocking free-to-paid conversion opportunities).

Higher impact means a higher RICE score:

- Massive = 3x
- High = 2x
- Medium = 1x
- Low = 0.5x
- Minimal = 0.25x

Confidence How well do we understand the customer problem? How well do we understand the solution and implementation details? Higher confidence means a higher RICE score.

- High = 100%
- Medium = 80%
- Low = 50%

Effort How many person months do we estimate this will take to build? Lower effort means a higher RICE score.

Calculating RICE Score

These four factors can then be used to calculate a RICE score via the formula:

$$(\text{Reach} \times \text{Impact} \times \text{Confidence}) / \text{Effort} = \text{RICE}$$

Here is an example RICE calculation you can use to help prioritize work in your area. Feel free to embed this at the Epic level to provide context for why you did or did not prioritize.

RICE Factor	Estimated Value
-----	-----
Reach	10.0
Impact	.5
Confidence	80%
Effort	2 month
-----	-----
Score	$(10.0 \times .5 \times .80) / 2 = 2.0$

Other important considerations:

- Is this in support of a company or team OKR?
- Does it bring our vision closer to reality?
- Does it help make our community safer through moderation tools?

- Does it meaningfully improve the user experience of an important workflow?
- Is it something we need ourselves?
- Is it particularly important to customers?
- The technical complexity is acceptable. We want to preserve our ability to make changes quickly in the future so we try to avoid complex code, complex data structures, and optional settings.
- It is orthogonal to other features (prevents overlap with current and future features).
- The requirements are clear.
- It can be achieved within the scheduled milestone. Larger issues should be split up, so that individual steps can be achieved within a single milestone.
- Refer to research participant gratuities section to understand if your study qualifies for incentive distribution.

We schedule a prioritized issue by assigning it a milestone; for more on this see Planning a Future Release.

Async RICE Exercise

Conducting a RICE prioritization exercise with your cross-functional counterparts is a powerful way to make the process more inclusive and improve the quality of your rankings. Consider making this an async-first process to accommodate team members across different timezones. For an example of how to do this async-first, see this issue that the Geo team used to collaborate on a RICE prioritization exercise. This blank async RICE template is also available for you to copy for your own async prioritization exercise.

Issues important to customers

For prioritizing most issues, we should utilize the RICE framework noted above, which will capture an aggregate of customer demand. You can also augment RICE scores with the Customer Issues Prioritization Framework Dashboards:

Customer Requested Issues (Product) for product managers
 Customer Requested Issues (CSM) for Sales, CS and CSM

These dashboards provide several inputs for calculating RICE and aggregate all customer requested issues and epics into a single dashboard. These dashboards are not meant as a replacement or sole input for Top ARR Drivers for Sales/CS. Further requirements such as the integration of themes need to be implemented before this framework can be used to fully inform or replace tools such as the Top ARR tracker.

In some cases however, we may become aware of a feature which is particularly important to deliver on by a certain date. Examples of this could include an issue necessary to embark on a new GitLab rollout, a feature needed by a partner to launch an integration, or a method to import data from a service which is being discontinued. In these instances, the responsible PM can apply the customer or customer+ label along with a due date and initial milestone. This set of labels can serve to indicate externally that the issue is particularly important, as well as a reminder for internal teams of its importance.

It is important to note that the customer and/or customer+ label does not constitute a promise for the issue to be delivered in any given milestone or time frame.

Community Considerations

GitLab is open-source, encouraging and promoting a large ecosystem of contributors is critical to our success. When making prioritization decisions, it's important to heavily weight activities which will encourage a stronger community of contributors. Some of those activities are:

- The creation of small primitives that can be utilized and iterated on by community members
- The building of integration points which can entice independent third parties to contribute an integration
- The addition of tools or features which make the contribution experience easier

Product managers are not responsible for prioritizing contributions outside of their group. These contributions should be reviewed and merged swiftly allowing everyone to contribute, including non-product teams at GitLab.

SaaS-First Framework

The SaaS-First product investment theme will put us in a better position to support our customer base who is expected to accelerate adoption of SaaS products in the coming years. Features will also end up more secure, resilient, performant, and scalable for our self-managed customers if initially built to the expectations of SaaS. Therefore, it is important for PMs to understand and prioritize needs related to the SaaS business. When prioritizing SaaS related issues, we follow the same guidelines above. Within those guidelines there are a few areas that are especially important for PMs to focus on to ensure the success of our SaaS users.

Availability

Downtime of GitLab.com has a material impact on our customers. From a 2014 report Gartner estimates that downtime costs companies on average "\$5,600 per minute, which extrapolates to well over \$300K per hour." Furthermore, SaaS downtime can severely disrupt the productivity of GitLab Inc since we rely heavily on GitLab.com to run our business. Finally, downtime can also lead to customer churn and damage to our reputation. Thus, it is crucial as a company we collectively work towards consistently maintaining our 99.95% SLA on GitLab.com. There are a few things that PMs can do in partnership with their engineering team to help ensure overall Availability for GitLab.com.

- Make sure each new feature that gets built has full end-to-end test coverage.
- Before rolling out a new service to support a major new feature launch, ensure that your team has gone through the readiness review process. The effort and timing for a readiness review will vary depending on the complexity of the feature. It is recommended to start this process as early as practical when a significant number of the questions can be answered but not too late to further develop the feature based on learnings from the review.
- Ensure there are application limits for your product areas enabled on GitLab.com to reduce abuse vectors.

Infradev

The infradev process is used to triage issues requiring priority attention in support of SaaS

availability and reliability. As part of the broader effort to responsibly manage tech debt across the company, PMs should partner with their EMs to identify and incorporate infradev labeled issues of all severities. Note, issues labeled with a severity must be mitigated and resolved within specific time frames to meet the SLO. As EMs are the DRIs for prioritizing infradev work, PMs should familiarize themselves with the infradev process and Board.

Other resources PMs can consult to identify and prioritize Infradev issues include:

- Error Budgets and associated dashboards
- The weekly triage report emailed to PMs now includes infradev items to prevent missing SLOs.
- Largest Contentful Paint metrics to monitor application performance.

While not required, PMs are encouraged to listen in on Incident Management calls for incidents related to their product areas to 1) build empathy with the SRE team by gaining insight into how they handle incidents 2) gain a better sense of the impact of the incident to their customer base, and 3) identify improvements to their product areas, whether technical or feature-related, that could have prevented the incident. PMs are not expected to be in the decision-making path on actions taken to resolve the incident. They are there to listen and learn rather than attempting to decide/influence the course of resolution. After incidents involving their product area, PMs are also encouraged to engage in the Incident Review, including attendance at the Sync Incident Review call if their incident is scheduled. PMs can periodically review incidents via the Production Incident Board

Enterprise Customer Needs

Enterprise customers interested in adopting SaaS may have common hard requirements to be able to use the product. For example, large enterprises may need certain security related features, such as Audit Logs, available before their security team will agree to the use of GitLab.com. This can also be about more than just features; it may include how and where we apply features so they can administrate their GitLab instance at enterprise-scale. For instance, permission management and shared configurations are best implemented top-down first instead of Project-up to meet the requirements of large organizations who may have 100s or 1000s of projects and only a small handful of people to perform these system-wide administrative tasks. In order to encourage more Enterprise adoption of GitLab.com, prioritize these common "hard-blockers" to adoption over "nice to have" features. PMs can use customer interviews to hone in on which issues are hard blockers to adopting SaaS vs more "nice to have" features that can be delivered later.

To track hard adoption blockers, use the ~"GitLab.com Enterprise Readiness" label within the GitLab-Org and GitLab-com groups.

SaaS Features

There are a few special considerations when it comes to delivering features for SaaS. In order to achieve parity between SaaS and Self-managed installations PMs should prioritize efforts to eliminate existing feature gaps that exist across the two installations. Additionally, new features should ship for SaaS and self-managed at the same time. Features should be implemented at the group level first, before being implemented at the instance level, so that they will work across both self-managed and SaaS. Finally, in order for new features to be adequately monitored, they should include appropriate logging and observability, which makes

troubleshooting much easier.

Working with Your Group

As a product manager, you will be assigned as the stable counterpart to a single group. At GitLab we abide by unique, and extremely beneficial guidelines when interacting with our groups. These include:

1. Product managers are the DRIs for overall work prioritization but work collaboratively with their EM, UX, and QEM stable counterparts to ensure the right priorities from each work type are considered as each has a different DRI. Product Managers are responsible for communicating overall priority.

1. Product Managers provide the what and when for feature work. Engineering (UX, Backend, Frontend, Quality) provide the how. This process is documented as part of our monthly product, engineering and UX cadence. We define stable counterparts for each of these functions within a group.

As an all-remote company, our crispness when it comes to responsibilities throughout the Product Delivery process was born out of necessity, but it pays untold dividends. Some of the benefits include:

- We avoid the ambiguity in handoffs between teams
- We avoid the confusion of many responsible individuals
- We avoid the slowness of consensus driven decision making
- We avoid the disruption of frequent context switching
- We gain the rigidity to be consistent
- We gain the freedom to iterate quickly

From Prioritization to Execution

As described above, prioritization is a multi-faceted problem. In order to translate the priorities of any given group into action by our engineering teams, we need to be able to translate this multi-faceted problem into a flat list of priorities for at least the next release cycle. Product Managers are responsible for taking all these prioritization considerations and creating a clear, sequenced list of next priorities. This list should be represented as an issue board so that each team has a clear interface for making decisions about work. From this list, Product Designers, Engineering Managers and Product Managers can work together to determine what items will be selected for work in the immediate future.

This does not mean that items will be addressed in strict order - Product Designers, EMs and PMs

need to be cognizant of dependencies, available skill sets, and the rock/pebbles/sand problem of time management to make the best decisions about selecting work.

Reviewing Build Plans

Together with your Engineering Manager, you will have an important role in ensuring that the Build Plans defined for issues are created with iteration in mind. Iteration is highly valuable for the following reasons:

1. It can result in discovering ways to parallelize effort, resulting in less team WIP and increase throughput
1. It can result in shipping something of value during an iteration rather than delaying everything
1. It can re-risk unknown unknowns by bringing them to light sooner in the development process

Prioritizing for Predictability {prioritize-predictability}

As a company we emphasize velocity over predictability. As a product manager this means you focus on prioritizing, not scheduling issues. Your engineering stable counterparts are responsible for velocity and delivery. However, there are instances when there is desire for predictability, including:

- Security, Bugs and Infra priorities with SLOs
- Customer Commitments
- Infrastructure projects with IACV driver impact or those that result in significant cost savings for gitlab.com
- Infrastructure projects with customer commitment or heavily upvoted should be given a priority indicative of other customer commitments
- Vision or Direction items for a launch

As the DRI for milestone prioritization, it is the Product Manager's job to prioritize for predictability when it is needed. You should do so by ensuring you prioritize a deliverable, and its dependencies, so that it can reasonably be expected to be delivered by any committed dates. If there is time pressure to hit a date, the PM should also explore de-scoping the issue to meet the deadline, rather than pressuring engineering to move abnormally fast or cut corners.

Private tools and dashboards for monitoring and KPI tracking

These information sources may be useful to help you prioritize.

- Feature usage
- EE usage: dev.gitlab.org account
- Grafana: Google gitlab.com account
- Kibana: dev.gitlab.org account
- S3stat: GitLab 1Password account
- Sentry: dev.gitlab.org account

Global Prioritization {prioritize-global}

Individual product managers must consider, and advocate for global optimizations within the teams they are assigned to. If your assigned team requires expertise (remember everyone can contribute)

outside the team you should make all reasonable efforts to proceed forward without the hard dependency while advocating within the product management team for increased prioritization of your now soft dependencies.

Execution of a Global prioritization can take many forms. This is worked with both Product and

Engineering Leadership engaged. Either party can activate a proposal in this area. The options available and when to use them are the following:

- Rapid action - use when reassignment isn't necessary, the epic can have several issues assigned to multiple teams
- Borrow - use when a temporary assignment (less than 6 months) to a team is required to help resolve an issue/epic
- Scope Reassignment - use when scope that will take longer than 6 months to deliver is a high priority and the team member reporting structure does not need to change to accomplish the effort.
- Realignment - use when a permanent assignment to a team is required to resolve ongoing challenges. This has the highest impact to team members and should be considered if other options cannot achieve the desired goal. We strive to hire team members in the groups that will need them most.

We have found the following methods less successful in ensuring completion of work that warrants global prioritization:

- Working Groups - This method involves convening a group of individuals who maintain full-time responsibility to other Product Groups and completing work as part of the working group structure. This method isn't preferred for completing product improvements, instead it can be utilized to scope work, or determine plans for future product delivery.
- Fan Out Prioritization - This method of prioritization involves communicating a global prioritization to a number of Product Groups in an effort to ensure each individual product group's PM prioritizes the work in the time frame you'd prefer. This method requires significant coordination costs and puts delivery at risk due to the lack of central prioritization responsibility. In most cases it is preferred to execute a scope reassignment, borrow or realignment to complete the improvements.

Planning and Direction

As a PM, you must plan for the near term milestones (more detailed) as well as for the long term strategy (more broad), and everything in between.

While monthly milestone planning is done in GitLab, longer horizon planning (1-3 years) is done in direction pages.

This will enable you to efficiently communicate both internally and externally how the team is planning to deliver on the product vision.

Managing your Product Direction

Documenting a Section, Stage, Group and Category direction is critical to communicating where we are heading and why to all of our stakeholders. This is especially important to the members of your Product Group. Establishing a direction for stakeholders (including team members) to participate in, and contribute to ensures there is a concrete connection to "Why" we are iterating and how it furthers GitLab's mission. Here are some of those connections:

- Improving Product Performance Indicators - Usage represents market capture (whether paying or not), and the start of our dual fly-wheel. For existing customers that market capture in new capabilities also represents increased retention and because of the benefits of a single application - user satisfaction.

- Improving Competitiveness against alternative DevOps tools - Leads to increased Stages Per user, and sales as they add to our "Increase Operational Efficiency"

As a Product Manager you can highlight these connections in:

- Direction Content and Overview Videos
- Weekly Meetings
- Individual Issue Descriptions
- Planning Issues
- Kickoff Videos
- Customer Discovery Interview Summaries

Communicating this connection requires a multi-channel approach. We should strive to share and communication about the connection to our Direction warrants consistent reinforcement.

Section and Stage Direction

Section leaders are responsible for maintaining Direction pages that lay out the strategy and plan for their respective section and stages. The direction pages should include topics outlined in this template.

Category Direction

A category strategy is required which should outline various information about the category including overall strategy, status, what's next, and the competitive landscape. The category strategy should be documented in a handbook page, which allows for version control of the category strategy as well as the ability to embed video assets. One of the most important pieces of information to include in the category strategy is a tangible next step or MVC and a clear description of focus and out-of-focus/maintenance areas.

Your category strategies should contain short paragraphs with lots of references to specific epics and issues.

Referencing topics, instead of features is encouraged as it's more stable over time.

We use this category strategy template as the outline for creating the handbook pages. If additional headings are needed you are empowered to create and populate them in your category strategy. You must keep these categories in sync with categories.yml and for new categories.

Category direction should be reviewed on a regular basis (at least monthly) by the responsible product manager. To indicate the last time a category direction page was reviewed, please ensure pages include Content Last Reviewed: yyyy-mm-dd at the top of the category content. Update this date with every review, even if other content on the direction page has not changed.

You should link to your category strategy from your stage strategy page.

For categories that have already shipped, and that have a marketing product page, categories.yml should link to the product page.

Inside of the categories.yml file there are dates assigned for either achieved or anticipated maturity achievement. These should be kept inline with communicated dates for achievement and updated as required.

If the category has developed a UX Roadmap we recommend the product designer to create a merge request to incorporate UX Roadmap themes into the category direction page roadmap. Assign the MR to the PM for review and merge.

Navigating cross-stage or cross-section direction pages

In some cases there may be direction pages that span multiple stages or sections. A direction page that summarizes the collective vision as well as all the contributors of that direction is critical to maintain transparency and adequate assignment of ownership.

There are several examples of these types of direction pages today:

- 1. Software Supply Chain Security Direction
- 1. AutoDevOps Direction
- 1. Monorepo Product Direction
- 1. Versioned Dependencies Direction
- 1. Customizable Dashboards Direction

The steps for creating and managing a cross-section or stage direction are:

- 1. Create a direction page merge request adding the direction page to the GitLab direction directory
- 1. Select the category change template in the merge request
- 1. Follow the process for category changes
- 1. Add CODEOWNERS by adding an entry with the direction page link and the page DRI GitLab Handle.
- 1. Once approved, @ all relevant product managers on the addition

Once the direction page has been added, there needs to be an assigned DRI for maintaining monthly updates for the page. It is the DRIs responsibility to ensure the shared direction page is regularly reviewed and is up to date. This requires cross-section / cross-stage collaboration from the DRI.

What makes a Product Direction issue?

You should use the ~direction label together with category and section labels to mark epics and issues that fall into the given direction.

Product Direction items (i.e., with the label) should be direction-level items that move the strategy forward meaningfully. This is up to the PM to set the bar for, but there should be a clear step forward with real user value.

It's important to note here that your plan is not simply a list of new features and innovation.

Those are included for sure, but so are issues related to all of your sensing mechanisms. A category upgrade from minimal to viable or delivery of a top customer issue (for example) can contribute to your plan just as much as a brilliant new innovative feature can. It's up to PMs to balance this through a coherent longer-term strategy. Conversely, in a broad sense anything could move the plan forward in a general way.

Finally, issues are the substance of your plan. Ensure you are applying the label to both relevant epics and its issues.

Communicating dates

As product managers, a core job is to set the correct expectations. We do this typically through discussing our direction and assigning issues to milestones. When you need to communicate specific dates, it's recommended doing it with limited visibility internally or directly to the customers. When you need to communicate specific dates use calendar year (CY) dates. Fiscal year (FY) does not translate well outside the company.

Accordingly, the direction pages are expected to refer to specific issues only for the next 3-4 months. Everything beyond that should discuss the topic, not specific issues.

Planning is indispensable but adjust, iterate

Creating a thoughtful direction for your section, stage, or category is a useful thought exercise that can help focus efforts, aid in prioritization, and get large groups of people on the same page. But beware of simply executing your long term plan. Our industry is incredibly dynamic, and we learn new things every day that can and should cause us to re-think our long term plans.

Delivery follows discovery

We should ship what brings value to our customers, not what is easy to ship. Stay focused on creating value each and every milestone, and be quick to adjust your longer term direction as you learn more.

1. When working on a larger theme, you should start with validating the end state knowing that it will change as you start shipping features and you learn more from actual usage.
1. Once the final vision is validated, you should work with your designer and engineering counterparts to break it down to the smallest possible iterations in order to ship value quickly.
1. You might still prefer to validate the first "milestone" before getting into delivery.
1. It's totally fine to never ship the initial vision and refine the vision after every iteration. A feature not built is much more valuable than a feature that is built but never used.

Maturity Plans

For each category, we recommend tracking the improvements required to advance to the next level of maturity. You are welcome to track maturity plans either with ~maturity:.... labels or maturity issues.

Maturity plans are highly encouraged - but not required - for non-marketing categories.

Planning and OKRs

GitLab users quarterly OKRs that cascade into Product OKRs and product group OKRs. You should have plans for the next three months in terms of driving specific product metrics through discovery and delivery actions.

You should discuss the product metrics with your manager, your design and engineering counterparts and the actions to reach the results with your design and engineering counterparts.

You can read more about the OKR process at GitLab at the two links shared above.

Planning Issue for Milestone

For each milestone, the planning quads come together to scope and plan work for the group for the upcoming milestone. Planning begins asynchronously with the creation of the planning issue. The planning issue is the SSOT for communication and all resources that are needed to plan a successful milestone. There are many ways to achieve to plan a milestone that should be curated based on the needs of the team. Below are a few examples of planning issues from groups across R&D to aid you in creating one that works best for your team.

- Verify::Pipeline Execution
- Verify::Testing
- Monitor::Respond
- Create::Code Review
- Create::Editor
- Data Stores::Database

As you adapt your own issue, it is recommended you apply the label planning issue to aid in tracking and to incorporate our Product Principles into the process.

Managing Upcoming Releases

Refer to the Product Development Timeline for details on how Product works with UX and Engineering to schedule and work on issues in upcoming releases.

Planning for Future Releases {planning-future-release}

There are two non-exclusionary ways to plan and communicate work for future releases

Planning with boards

As a Product Manager you can maintain prioritization of your groups issues using a fully prioritized issue board where the ordering of the issues reflects their priority.

Planning with milestones

Product Managers can assign milestones to issues to indicate when an issue is likely

to be scheduled and worked on.

Still, whether an issue can be delivered within a milestone is the decision of the engineering team.

As we consider more distant milestones, the certainty of the scope of their assigned issues and their implementation timelines is increasingly vague. In particular, issues may be moved to another project, disassembled, or merged with other issues over time as they bounce between different milestones.

The milestone of an issue can be changed at any moment. The current assigned milestone reflects the current planning, so if the plan changes, the milestone should be updated as soon as possible to reflect the changed plan. We make sure to do this ahead of starting work on a release. Capacity is discussed between the PMs and the engineering managers.

There are helper labels to signal these plans like ~next::1-3 releases and its variants.

Special milestones

In addition, we have two special milestones: Backlog and Awaiting further demand. Product Managers assign these issues to milestones that they have reviewed and make sense, but do not fit within the upcoming release milestones due to either a lack of comparative urgency or because we have not yet seen enough user demand to prioritize the item yet. The best way to demonstrate urgency on either of these items is to vote on them and, if possible, add comments explaining your use case and why this is important to you.

Recommendation for when to change 'Awaiting further demand':

Always focus on the overall value of the feature.

Do you have a good understanding of the user problem?

Do you have a good understanding of the impacted user base?

Was the proposed solution validated?

Issues with the 'Awaiting further demand' label often mean poorly understood requests that require more information from our users and the market.

Often public feedback only comes from a small percentage of people using or evaluating a feature or product.

You should always consider reaching out directly to our users to learn more about their use cases.

Recommendation when changing a previously planned issue to Backlog: When moving a previously planned issue to Backlog, especially one planned for within the next release or two,

consider the message that this may be sending to parties that were interested in this feature.

In some cases, they may have been depending or planning upon the issue to be delivered around the assigned milestone, and with the change to Backlog that is now unlikely to occur. In these instances,

it is best to concisely explain the rationale behind the change in a comment, so the community can understand and potentially respond with additional justification or context. It is also encouraged to move the issue to the Backlog as soon as it is clear that it will not be scheduled in the near future. This will help with understanding the change, as it

will not seem like a last minute change.

Communicating clearly changing priorities might encourage the community to contribute the issue to GitLab.

Again, the milestone of an issue can be changed at any moment, including for both of these special milestones.

Shifting commitment mid-iteration

From time to time, there may be circumstances that change the ability for a team to ship the features/issues they committed to at the beginning of the iteration.

These steps also apply when an issue is broken into multiple issues.

When this happens, as a PM you must coordinate with your EM counterpart that the impacted issues and their milestones are updated to reflect the new reality (for example, remove deliverable tag, update milestone, etc.). Additionally, notify your manager of the shift.

Utilizing our design system to work autonomously

Our design system provides the means to work autonomously, without always needing UX insight, feedback and design. When problems can be solved using an already documented paradigm, you don't need to wait for UX approval to bring an issue to a reasonable state within a first iteration.

If lingering questions remain, subsequent iterations can address any shortcomings the feature might have.

Always consider that with a dedicated product designer, it's much faster and cheaper to iterate on a design than to re-implement it.

At the same time, not everything needs a design, and the design system is here to support your engineers and you in those cases.

Iteration Strategies

Iteration is a core value of GitLab, and product management has a central role to play in it. Iteration should be apparent as we deliver new features in MVCs, but it has implications for discovery too. As solution validation can move much faster than delivery, we should aim to validate features before building them. At this point, the feature validated is likely way bigger than an MVC if we would build it. We should pay special attention as product managers to still aim at iterative delivery after a bigger feature-set got validated, as delivered features provide the final validation. For example, once a direction is validated, we can start the delivery by documentation. As product managers we should aim to iterate as part of solution validation, and while delivering already validated solutions too.

Here are several strategies for breaking features down into tiny changes that can be developed and released iteratively. This process will also help you critically evaluate if every facet of the design is actually necessary.

Workflow steps

As part of design and discovery, you likely created a minimal user journey that contains sequential steps a user is going to take to "use" the feature you are building. Each of these should be separated. You can further by asking yourself these questions:

- Can/is it desirable to perform this action via the UI or can we use a non-UI approach as a start (for example, CLI, API or .csv download of data)? This is a great starting point before adding UI components that achieve the same thing.
- Will there be different UI paths to perform the same task? Identify which are the most useful and which are the easiest to implement. Weight both factors when determining which to start with, and build from there.

User operations

View, Create, Update, Remove and Delete are actions users take while interacting with software. These actions naturally provide lines along which you can split functionality into smaller features. By doing this, you prioritize the most important actions first. For example, users will likely need to be able to visually consume information before they can create, update, remove, or delete.

Functional criteria

Often, the criteria for features are built on is implicit. It can help to use a test-driven development mindset where you write the tests and the outcomes you need from the software before building the software. Writing these tests can uncover the different criteria you need the development team to meet when building the new feature. Once you've outlined these tests, you may be able to use them to continue to break down the feature into smaller parts for each test. Here are a few examples:

- What is the default behavior when there is no data (empty/null state)?
- Are there automatic actions or events that occur as part of your feature? Write them down, and identify those that can be done manually by the user before adding automation.
- Will users of different roles have unique experiences? Can you prioritize and build one of these experiences first? (for example: guest, user, developer, maintainer).
- Do users want to be able to customize their view of information? Define all of the customizations you want to offer, and build them one at a time (for example, toggle on/off, filter, sort, search).

Exception & error cases

Software often fails and can fail in different ways depending upon how it is architected. It is always best to provide the user with as much information as possible as to why something did not behave as expected. Creating and building different states to handle all possible errors and exceptions can easily be broken down into individual issues. Start by creating a generic error state to display when anything goes wrong, and then add on to handle different cases one by one. Remember to always make error messages useful, and add additional error messages as you identify new error states.

Breaking down the UI

Breaking down a design into pieces that can be released iteratively is going to depend on what

you are building. Here are a few helpful questions to guide that process:

- What components already exist that you can reuse to go faster?
- What constitutes "extra styling"? Is there a way to display the information you need to display plainly and then add details later?
- Do you have lots of interactions in the design that make the UX lovable? Can you pull those out into separate issues and add them iteratively? (e.g. hover states, dra